



SERVERLESS IMAGE PROCESSING WITH AWS LAMBDA

- KIDUS MIKAEL BIRHANU
- SOHAN ARUN
- SREEDHARANI MACHUNURU

INTRODUCTION

- Objective of the Project
 - Deploy an image processing application on AWS Lambda
 - Demonstrate scalability using CloudWatch metrics
 - Leverage serverless computing for real-time processing
- Expected Outcomes
 - A scalable, cloud-based image processing pipeline

WHAT IS AWS LAMBDA?

- AWS Lambda is a serverless computing service that automatically runs code in response to triggers. It scales automatically based on demand, making it ideal for real-time applications like image processing.
- Why Use AWS Lambda?
 - ✓ Serverless – No need to manage servers.
 - ✓ Auto-Scaling – Scales up and down based on requests.
 - ✓ Event-Driven – Can be triggered by S3, API Gateway, DynamoDB, etc.
 - ✓ Cost-Effective – You only pay for the time the function runs.
- How It Works in This Project:
 - API Gateway triggers the function via HTTP requests.
 - The function processes images (converts them to grayscale).
 - Processed images are returned as base64 and stored in S3.
 - CloudWatch monitors scaling behavior.

SOLUTION OVERVIEW

Technology Stack Used

AWS Lambda

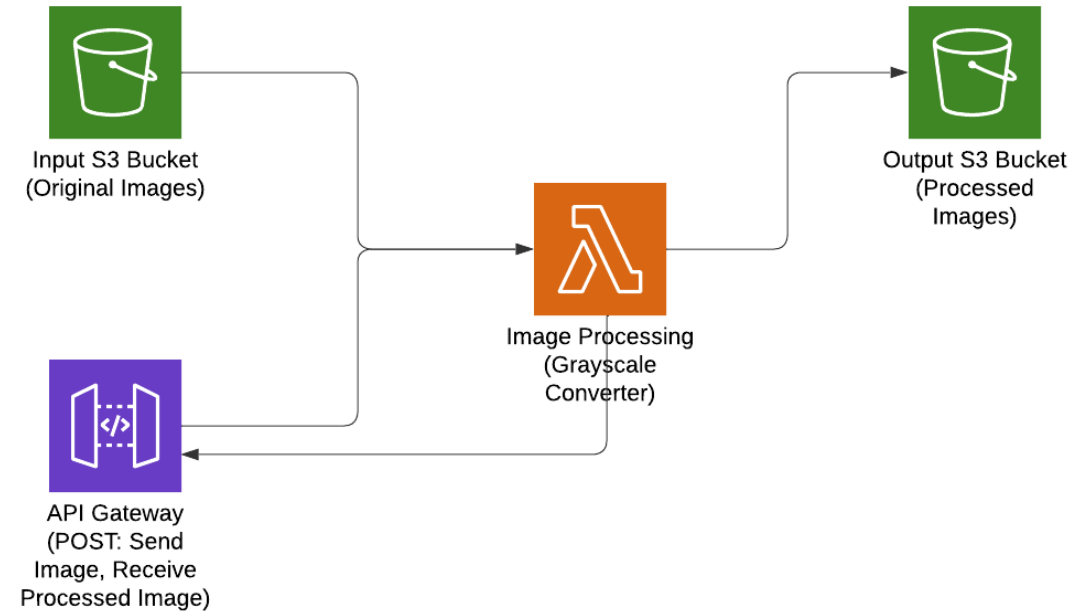
S3 (Storage)

API Gateway

OpenCV for image processing

CloudWatch for monitoring

Postman to send multiple requests



PREPARING OPENCV FOR AWS LAMBDA

1. Checked Python Version ubuntu VM – Ensured compatibility with AWS Lambda runtime Python version (12.3).
2. Installed pip3 – Required for managing dependencies like OpenCV.
3. Created Installation Directory – Followed AWS Lambda's required directory structure (site-packages).
4. Installed OpenCV – Used opencv-python-headless to avoid GUI dependencies.
5. Zipped OpenCV Package – Prepared it as a .zip file for AWS Lambda Layers.
6. Uploaded to S3 – Since AWS Lambda has a size limit for deployment packages, using S3 allows Lambda to retrieve and use the OpenCV package efficiently.
7. Created Lambda Layer – Made OpenCV reusable across multiple Lambda functions.

SETTING UP THE LAMBDA FUNCTION

- Lambda Function Responsibilities
- Read an image
- Convert it to grayscale using opencv
- Store the output in a destination S3 bucket
- Initial Trigger Mechanism: S3 PUT Event
- Function gets invoked on image upload

CHALLENGES WITH S3 TRIGGERS & TRANSITION TO API GATEWAY

- Issues with S3 PUT Triggers
- Inefficient for testing scalability
- Lack of control over request volume
- Solution: API Gateway as a Trigger
- Enabled HTTP-based image processing
- Allowed sending image data via API requests as a base 64 text with the request

IMPLEMENTING API GATEWAY WITH AWS LAMBDA

- Modified Function to Accept HTTP Requests
- Accepts base64-encoded image in request body
- Decodes, processes, and returns grayscale image with the response of the request and adds the edited image to the destination S3 bucket
- Advantages of API-based Processing
 - Enabled Effective Scaling Tests – Allowed us to repeatedly trigger the Lambda function to observe its scaling behavior.
 - More Efficient Than S3 PUT Triggers – Uploading multiple images to S3 was not an effective way to generate high-frequency function invocations.
 - Allowed Controlled Concurrent Requests – Made it easier to send a high volume of requests programmatically using Postman.

PERFORMANCE TESTING — LAMBDA SCALING

- Load Testing with Postman
 - Sent HTTP requests in varying loads
 - Requests tested: 1, 3, 5, 20, 50, 100, 80, 70, 50
 - Used Postman pre-request script to send multiple requests concurrently via a JavaScript loop.
- Metrics Monitored in CloudWatch
 - Total Concurrent Invocations (Auto-scaling behavior).
 - Error and Success Rate (Ensuring function stability).
 - Execution Duration (Performance under different loads).
 - Invocation Count (Tracking total function calls).

Total concurrent executions

1 minute ▼ Maximum ▼ 1h 3h 12h 1d 3d 1w Custom Local timezone



Count



Total concurrent executions

View in metrics

Close

Total concurrent executions

1 minute ▼ Maximum ▼ 1h 3h 12h 1d 3d 1w Custom UTC timezone



Count



Total concurrent executions

View in metrics

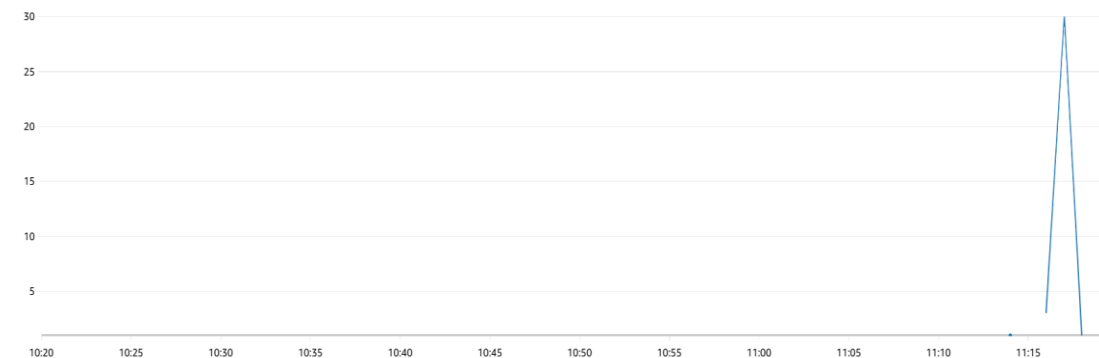
Close

Total concurrent executions

1 minute ▼ Maximum ▼ 1h 3h 12h 1d 3d 1w Custom Local timezone 10 seconds



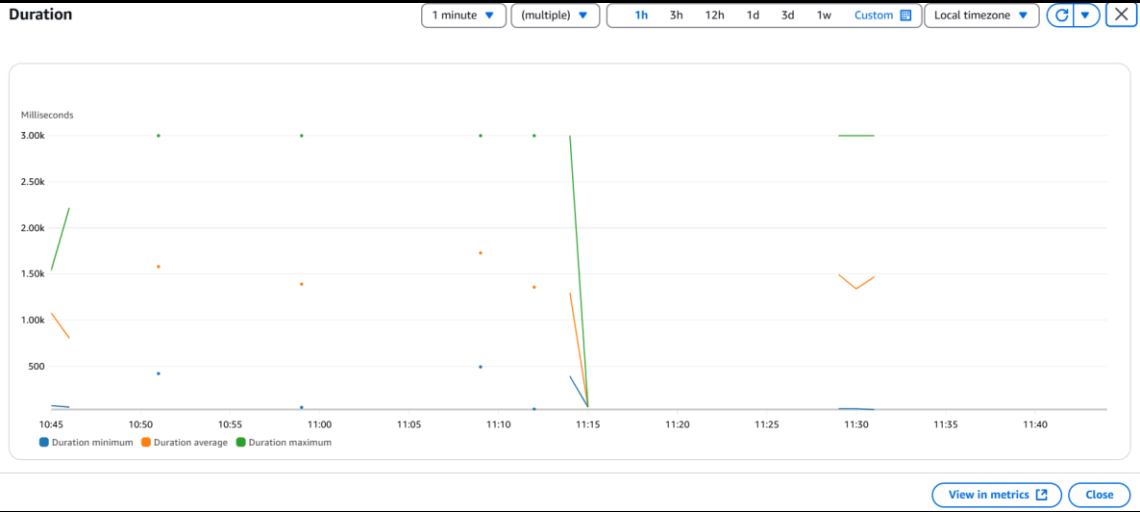
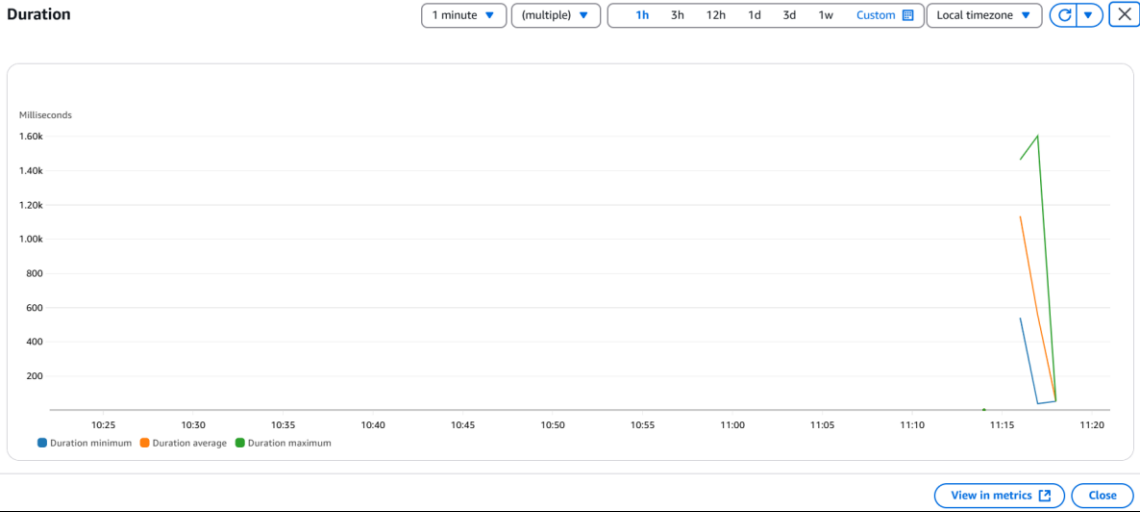
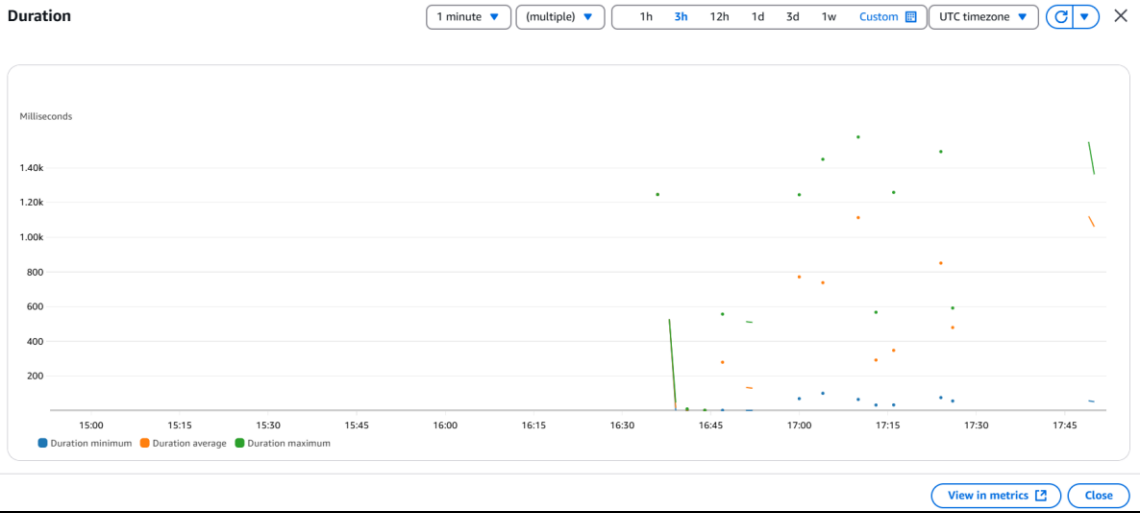
Count



Total concurrent executions

View in metrics

Close



OBSERVING AUTO-SCALING WITH CLOUDWATCH

- Lambda Scaled Up and Down Dynamically
 - AWS Lambda automatically adjusted the number of concurrent executions based on the incoming requests.
 - When requests increased, Lambda spawned more function instances, and when traffic decreased, it scaled down efficiently to optimize resource usage.
- Higher Requests = More Concurrent Function Executions
 - As we increased the number of concurrent HTTP requests (e.g., 20, 50, 100), we observed a corresponding rise in the number of active Lambda instances.
 - This confirms that Lambda dynamically provisions resources in response to demand.
- Scaling Efficiency Observed Through CloudWatch
 - CloudWatch metrics (Concurrent Invocations, Execution Duration) clearly showed the function adapting to traffic spikes.
 - The scaling response was quick and smooth, demonstrating the efficiency of serverless computing for handling varying workloads.

RESULTS & KEY TAKEAWAYS

- Lambda Scaled Efficiently – Automatically adjusted resources based on request volume, ensuring smooth performance.
- API Gateway Improved Testing – Allowed controlled concurrent requests via Postman, making scaling tests more effective than S3 triggers.
- CloudWatch Provided Key Insights – Monitored metrics like total Concurrent Invocations and Execution Duration, confirming auto-scaling.
- Serverless Enabled Real-Time Processing – Lambda's event-driven nature processed images on demand without managing infrastructure.

CONCLUSION

- Successfully deployed a scalable, serverless image processing application using AWS Lambda.
 - Demonstrated automatic scaling based on demand through CloudWatch metrics.
 - API Gateway, along with HTTP requests and Postman scripts, allowed efficient testing of Lambda's Serverless computing enabled real-time processing without infrastructure management.
-  Our project proves that AWS Lambda is an effective solution for dynamic, scalable cloud-based image processing.

The background is a dark teal color, densely populated with numerous speech bubbles of various colors including red, yellow, purple, and grey. Each speech bubble contains a large, dark blue question mark. The bubbles are scattered across the entire frame, creating a pattern that suggests a Q&A session or a forum.

THANK YOU, ANY
QUESTIONS